MOD -03 - JDK new Features

MOD -03 - JDK new Features

LUMINAR
TECHNOLAB

What is Functional Interface?

1. A Functional Interface in Java is an interface that contains exactly one abstract method. It can have multiple default and static methods, but only a single abstract method is allowed.
2. This only one method specifies the intended purpose of the interface.

3. Functional interfaces enable the use of lambda expressions and method references, making code more concise and readable.

For example, the Runnable interface from package java.lang; is a functional interface because it constitutes only one method i.e. run().

How to define a user defined Functional Interface in java

```
import java.lang.FunctionalInterface;
@FunctionalInterface
public interface InterfaceDemo{
    // single abstract method
    double getDetails();
}
```

@FunctionalInterface. The annotation forces the Java compiler to indicate that the interface is a functional interface. So, does not allow to have more than one abstract method.

Java 7 - Single Abstract Methods (SAM type)

In Java 7, functional interfaces were considered as Single Abstract Methods. SAMs were commonly implemented with Anonymous Classes in Java 7.

Pros : fewer codes in Java 7

Cons :the syntax was still difficult and a lot of extra lines of code were required.

In java 8 , a functional interface has just one method, there should be no need to define the name of that method when passing it as an argument. Lambda expression allows us to do exactly that.

lambda expressions

1. Lambda expression is, an anonymous or unnamed method.
2. The lambda expression does not execute on its own.
3. Instead, it is used to implement a method defined by a functional interface.

How to define lambda expression in Java?

(parameter list) -> lambda body

Note :

-> arrow operator or a lambda operator

Normal Method	Lambda expression

double getPiValue() {
return 3.1415;
}
	() -> 3.1415

1. double - return type 2. getPiValue - method name 3. () - method does not have any parameters 4. return 3.1415 – return value as double	1. The method does not have any parameters. So, the left side of the operator includes an empty parameter. 2. The right side is the lambda body that specifies the action of the lambda expression. In this case, it returns the value 3.1415.

Normal Interface Method implementation

```java
package functionalinterfacelambda;


interface NormalDemoInterface{

     // public abstract method
     double getPiValue();
}

public class NormalMethod implements NormalDemoInterface {
    @Override
    public double getPiValue() {
        return 3.1415;
    }

    public static void main( String[] args ) {

        //  A reference variable of a superclass can be used to a refer any subclass object .
        // Here we are assigning the object of the class to the reference variable of the interface

        NormalDemoInterface di=new NormalMethod();
        System.out.println("Value of Pi = " + di.getPiValue());
        }
}

```

Java 7 SAM Anonymous implementation

```java
package functionalinterfacelambda;

interface NormalDemoInterfaces{

     // public abstract method
     double getPiValue();
}

public class SamJavaSevenAnonymus {
    public static void main(String[] args) {
          NormalDemoInterfaces sam=new NormalDemoInterfaces() {
              @Override
                  public double getPiValue() {
                    return 3.14;
                  }
          };
          System.out.println(sam.getPiValue());
    }
}
```

Java 8 @ Functional Interface – Lambda expression

```java
package functionalinterfacelambda;

import java.lang.FunctionalInterface;

//this is functional interface
@FunctionalInterface
interface DemoInterface{

     // abstract method
     double getPiValue();
}
public class LambdaNoParameter {

    public static void main( String[] args ) {

        // declare a reference to MyInterface
        DemoInterface di;
        // lambda expression
        di = () -> 3.1415;
        DemoInterface dil=()->3.14;
        System.out.println("Value of Pi = " + di.getPiValue());
        System.out.println("Value of Pi = " + dil.getPiValue());
        }
}

```

Problems | Javadoc | Declaration | Console | Outline

```
<terminated> LambdaNoParameter [Java Application] C:\Program Files\Java\jdk-11.0.13\bin\javaw.exe
Value of Pi = 3.1415
Value of Pi = 3.14
```

Java 8 @ Functional Interface – Lambda with parameter

```java
package functionalinterfacelambda;

@FunctionalInterface
interface MyInterface {

    // abstract method
    String reverse(String s);
}
public class LambdaWithParameter {
    public static void main( String[] args ) {

        // declare a reference to MyInterface
        // assign a lambda expression to the reference
        MyInterface mi = (str) -> {
            String result = "";
            for (int i = str.length()-1; i >= 0 ; i--)
            result += str.charAt(i);
            return result;
        };

        // call the method of the interface
        System.out.println("String reversed = " + mi.reverse("Luminar"));
    }
}
```

Problems | Javadoc | Declaration | Console | Outline

```
<terminated> LambdaWithParameter [Java Application] C:\Program Files\Java\jdk-11.0.13\bin\javaw.exe
String reversed = ranimuL
```

Java 8 @ Functional Interface- Generic– Lambda with parameter

```java
package functionalinterfacelambda;

//functional interface generic. so that any data type is accepted

@FunctionalInterface
interface GenericMyInterface<T> {

    // abstract method
    T method(T t);
}

public class GenericFunctionalLambda {
    public static void main( String[] args ) {

        // Step 1: declare a reference to GenericInterface
        // Step 2: the GenericInterface operates on String data
        // Step 3: assign a lambda expression to it

        GenericMyInterface<String> reverseString = (str) -> {
            String result = "";
            for (int i = str.length()-1; i >= 0 ; i--)
            result += str.charAt(i);
            return result;
        };

        System.out.println("Lambda reversed = " + reverseString.method("Luminar"));

```

```java

        // Step 1: declare another reference to GenericInterface
        // Step 2: the GenericInterface operates on Integer data
        // Step 3: assign a lambda expression to it

        GenericMyInterface<Integer> factorial = (no) -> {
                int result = 1;
                for (int i = 1; i <= no; i++)
                result = i * result;
                return result;
            };

            System.out.println("Lambda reversed = " + factorial.method(5));

```

```java

            // Step 1: declare another reference to GenericInterface
            // Step 2: the GenericInterface operates on Integer data
            // Step 3: assign a lambda expression to it

            GenericMyInterface<Integer> reversNo = (num) -> {
                int m=0,rev=0;
                while(num>0) {
                    m=num%10;
                    rev=(rev*10)+m;
                    num=num/10;

                }
                return rev;
             };

                System.out.println("Lambda reversed = " + reversNo.method(567));

    }
}
```

Problems | Javadoc | Declaration | Console | Outline

<terminated> GenericFunctionalLambda [Java Application] C:\Program Files\Java\jdk-11.0.13\bin\javaw.exe

```
Lambda reversed = ranimuL
Lambda reversed = 120
Lambda reversed = 765
```

Thank you ☺ Happy learning ☺